Programowanie współbieżne w C++

Lista zadań nr 4

Na pracownię 26 marca 2026

Zadanie 1 (5pkt). Współbieżna kolejka priorytetowa o ograniczonej pojemności.

Zaimplementuj wielowątkową kolejkę priorytetową o ograniczonej pojemności, opakowując `std::priority_queue`. Kolejka musi mieć twardy limit pojemności (tzw. Bounded Queue) podawany w konstruktorze. Użyj dokładnie jednego obiektu std::mutex do ochrony operacji na strukturze oraz dokładnie dwóch obiektów `std::counting_semaphore` do usypiania i budzenia wątków producentów i konsumentów. Zaprojektuj interfejs tak, by jego użycie nie generowało wyścigów danych. Napisany przez Ciebie kod musi zapewniać silną gwarancję wyjątków. Pamiętaj, że skopiowanie własnego obiektu `T` podczas odczytywania go z kolejki może rzucić wyjątek (np. `std::bad_alloc` lub wyjątek rzucony celowo w konstruktorze kopiującym typu `T`). W przypadku wyrzucenia wyjątku:

- Element **nie może bezpowrotnie zniknąć** z kolejki.
- Semafory **nie mogą ulec rozsynchronizowaniu** z faktycznym rozmiarem kolejki.

Oto szkielet rozwiązania.

```
#include <queue>
#include <mutex>
#include <semaphore>
#include <memory>

template <typename T>
class BoundedPriorityQueue {
private:
    std::priority_queue<T> pq;
    std::mutex mtx;
    // Semafory do kontroli pojemności (liczba wolnych miejsc
i liczba elementów)
    std::counting_semaphore<> free_slots;
    std::counting_semaphore<> available_items;

public:
    explicit BoundedPriorityQueue(int capacity)
        : free_slots(capacity), available_items(0) {}
```

```
    void push(T value) {
        // MIEJSCE NA KOD
    }

    // Zadecyduj o sygnaturze zwracanej wartości (np. T lub
std::shared_ptr<T>)
    T??? wait_and_pop() {
        // MIEJSCE NA KOD
    }

    T??? try_pop() {
        // MIEJSCE NA KOD
    }

};
```

Zadanie 2 (6pkt). Współbieżny graf relacji.

Zaimplementuj graf, w którym wierzchołki posiadają własne, niezależne blokady. Głównym wyzwaniem jest wątkowo bezpieczne modyfikowanie i odczytywanie relacji (krawędzi) pomiędzy parami wierzchołków bez powodowania zakleszczeń, maksymalizując współbieżność odczytów. Dokładniej, zaprojektuj klasę `ConcurrentGraph`. Graf składa się z tablicy wierzchołków. Każdy wierzchołek (klasa `Vertex`) posiada:

- Swój unikalny identyfikator (np. int id, tzn. indeks wierzchołka w tablicy).
- Strukturę przechowującą identyfikatory sąsiadów (np. std::unordered_set<int>).
- Własny obiekt std::shared_mutex, który chroni dostęp do jego listy sąsiadów.

Dla uproszczenia (aby nie musieć blokować całej struktury przy dodawaniu nowych węzłów), załóżmy, że zbiór wierzchołków jest inicjalizowany raz w konstruktorze grafu i potem się nie zmienia (nie dodajemy/nie usuwamy samych węzłów, modyfikujemy tylko relacje między nimi). Graf powinien oferować następujące metody:

1. `void add_edge(int id_a, int id_b)` oraz `void remove_edge(int id_a, int id_b);`
   Działanie: Dodaje lub usuwa obustronną relację (krawędź nieskierowaną) między węzłem A i B. Wymaga modyfikacji list sąsiadów w obu wierzchołkach. Wątek musi uzyskać wyłączny dostęp (`std::unique_lock`) do obu wierzchołków jednocześnie.

2. `bool are_connected(int id_a, int id_b) const;`
   Działanie: Sprawdza, czy istnieje krawędź między A i B. Ta operacja wymaga jedynie odczytu. Użyj `std::shared_lock` na jednym z wierzchołków (np. A), sprawdź czy B jest na jego liście i zwolnij blokadę. Pozwala to innym wątkom na jednoczesne czytanie relacji węzła A.
3. `std::vector<int> get_common_neighbors(int id_a, int id_b) const;`
   Działanie: Zwraca listę wspólnych sąsiadów dla wierzchołków A i B. Zablokuj do odczytu węzeł A listę węzeł B **w tym samym czasie**, a następnie wyznacz część wspólną list ich sąsiadów.
4. `void print_node_stats(int id_a) const;`
   Działanie: Operacja symulująca dłuższe działanie niemodyfikujące na konkretnym węźle (np. iterowanie po jego sąsiadach, obliczanie statystyk, w celach testowych dodajemy sztuczne opóźnienie `std::this_thread::sleep_for`). Cel: Udowodnienie, że gdy jeden wątek analizuje wierzchołek A używając `shared_lock`, inne wątki mogą wciąż bez problemu wywoływać `are_connected` lub `get_common_neighbors` dla wierzchołka A (wielu czytelników), ale wywołanie `add_edge` dla A zostanie wstrzymane.

Wskazówka: Pamiętaj o możliwości wystąpienia zakleszczeń przy blokowaniu dwóch semaforów. Wyklucz tę możliwość!

Zadanie 3 (4+1pkt). Bariera wznawialna.

Mamy stałą liczbę N wątków. Bariera to punkt synchronizacji, w którym każdy z wątków czeka, aż wszystkie N dotrze do niego. Dopiero wtedy mogą przejść przez barierę i kontynuować wykonanie.

```
Barrier barrier(3); // Dla N = 3 wątki

void worker() { // funkcja wykonywana przez każdy z N wątków
  while (1) {
    // ... obliczenia ...
    barrier.arrive_and_wait();
    // ... kontynuacja ...
  }
}
```

Funkcja `arrive_and_wait()` na obiekcie bariery może być wykonywana wielokrotnie, zatem mamy do czynienia z tzw. barierą wznawialną (w odróżnieniu od bariery

jednorazowej/zatrzasku). Zaimplementuj klasę Barrier. Jedynym dozwolonym środkiem synchronizacji są muteksy i semafory zliczające (nie można używać zmiennych warunkowych). Dodatkowy punkt otrzymasz, jeśli Twoja bariera będzie oferować **funkcję zakończenia fazy** (CompletionFunction) na wzór std::barrier.